Universidad Politécnica Metropolitana del Estado de Hidalgo

Maestría en Inteligencia Artificial

Evolución Diferencial: Implementación, Estudio de Casos y Aplicaciones Prácticas

Algoritmos de optimización basados en poblaciones
para problemas continuos

Materia: Algoritmos Evolutivos
Alumna: Jessica Melani Romero Lora
Profesor: Dr. Jaime Aguilar Ortiz
Fecha: Octubre 2026

1. Introducción

La Evolución Diferencial (DE, por sus siglas en inglés) es un algoritmo de optimización metaheurístico propuesto por Storn y Price en 1995. Pertenece a la familia de algoritmos evolutivos y fue diseñado específicamente para resolver problemas de optimización con variables continuas en espacios de alta dimensionalidad. A diferencia de otros métodos como los algoritmos genéticos tradicionales, DE utiliza diferencias vectoriales entre individuos de la población como mecanismo de perturbación, lo que le confiere propiedades de auto-adaptación en cuanto a la magnitud y dirección de la búsqueda.

Su relevancia en la ingeniería y las ciencias aplicadas radica en tres factores: requiere pocos parámetros de control (F, CR y NP), no necesita información de gradiente y ha demostrado un desempeño competitivo frente a técnicas como Particle Swarm Optimization (PSO) y Covariance Matrix Adaptation (CMA-ES) en numerosos benchmarks internacionales.

1.1. Estructura del algoritmo

El flujo de DE consta de cuatro etapas iterativas que se repiten hasta satisfacer un criterio de paro: inicialización aleatoria de la población, mutación mediante diferencias vectoriales, cruzamiento binomial (o exponencial) para generar vectores de prueba y selección greedy que conserva al mejor entre el vector objetivo y el de prueba.

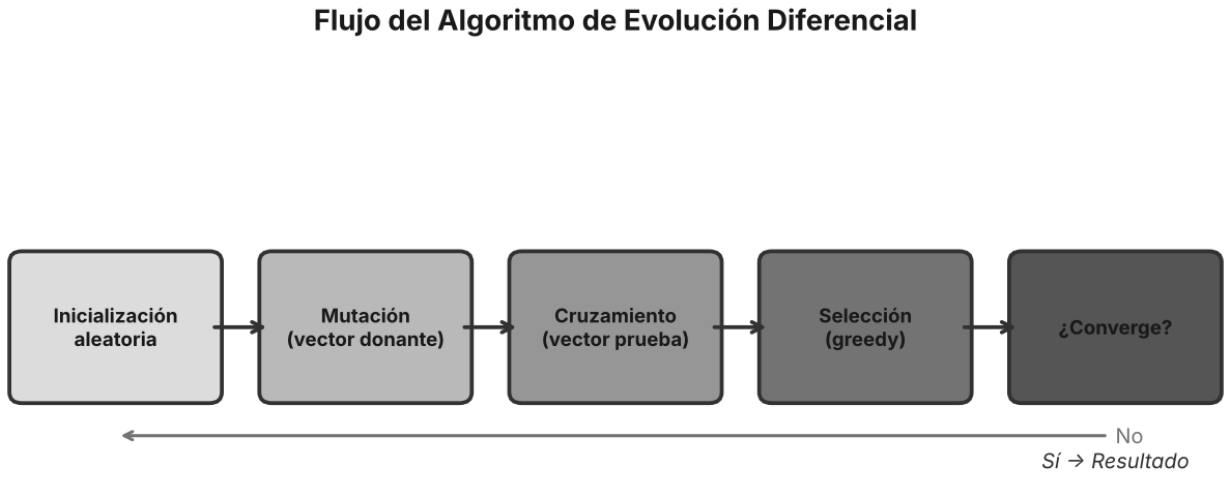


Figura 1. Diagrama de flujo del algoritmo de Evolución Diferencial.

1.2. Formulación matemática

Sea $P = \{x_1, x_2, ..., x_NP\}$ la población de NP vectores en $\mathbb{R}^D$. En cada generación g, para cada vector objetivo x_i se genera un vector donante (mutante) v_i y un vector de prueba u_i mediante:

Mutación (DE/rand/1): $v_i = x_r1 + F \cdot (x_r2 - x_r3)$

Cruzamiento binomial: $u_{i□} = v_{i□}$ **si rand < CR o j = j_rand,** $x_{i□}$ **en otro caso**

Selección: $x_i^{(g+1)} = u_i$ **si** $f(u_i) \leq f(x_i)$, x_i **en otro caso**

Donde r1, r2, r3 son índices distintos entre sí y distintos de i, seleccionados aleatoriamente; $F \in [0, 2]$ es el factor de escala; $CR \in [0, 1]$ es la probabilidad de cruzamiento; y j_rand es una dimensión seleccionada al azar para garantizar que al menos un componente del vector de prueba proviene del vector donante.

2. Principales Variantes

La nomenclatura estándar DE/x/y/z describe: x la base del vector donante (rand o best), y el número de diferencias vectoriales empleadas y z el tipo de cruzamiento (bin o exp). Cada variante equilibra de forma distinta la exploración y la explotación del espacio de búsqueda.

Principales Variantes de Evolución Diferencial

Variante	Fórmula de mutación	Característica
DE/rand/1/bin	v = x_r1 + F·(x_r2 − x_r3)	Alta exploración
DE/best/1/bin	v = x_best + F·(x_r1 − x_r2)	Convergencia rápida
DE/rand/2/bin	v = x_r1 + F·(x_r2−x_r3+x_r4−x_r5)	Mayor diversidad
DE/current-to-best/1	v = x_i + F·(x_best−x_i) + F·(x_r1−x_r2)	Equilibrio expl./expt.

Figura 2. Comparativa de las principales variantes de Evolución Diferencial.

DE/rand/1/bin es la variante clásica y la más utilizada en la literatura. Favorece la exploración al usar vectores base aleatorios. DE/best/1/bin sustituye el vector base aleatorio por el mejor individuo de la población, lo que acelera la convergencia pero incrementa el riesgo de estancamiento en óptimos locales. DE/rand/2/bin emplea dos diferencias vectoriales, generando perturbaciones de mayor diversidad. Por último, DE/current-to-best/1 combina la dirección hacia el mejor individuo con una perturbación aleatoria, logrando un equilibrio entre exploración y explotación que resulta eficaz en funciones multimodales.

2.1. Operación de mutación

La mutación es la operación central de DE que la distingue de otros algoritmos evolutivos. En lugar de generar perturbaciones con distribuciones predefinidas, utiliza la diferencia

entre vectores existentes de la población, lo que adapta automáticamente la magnitud y dirección de la búsqueda a la geometría del paisaje de aptitud.

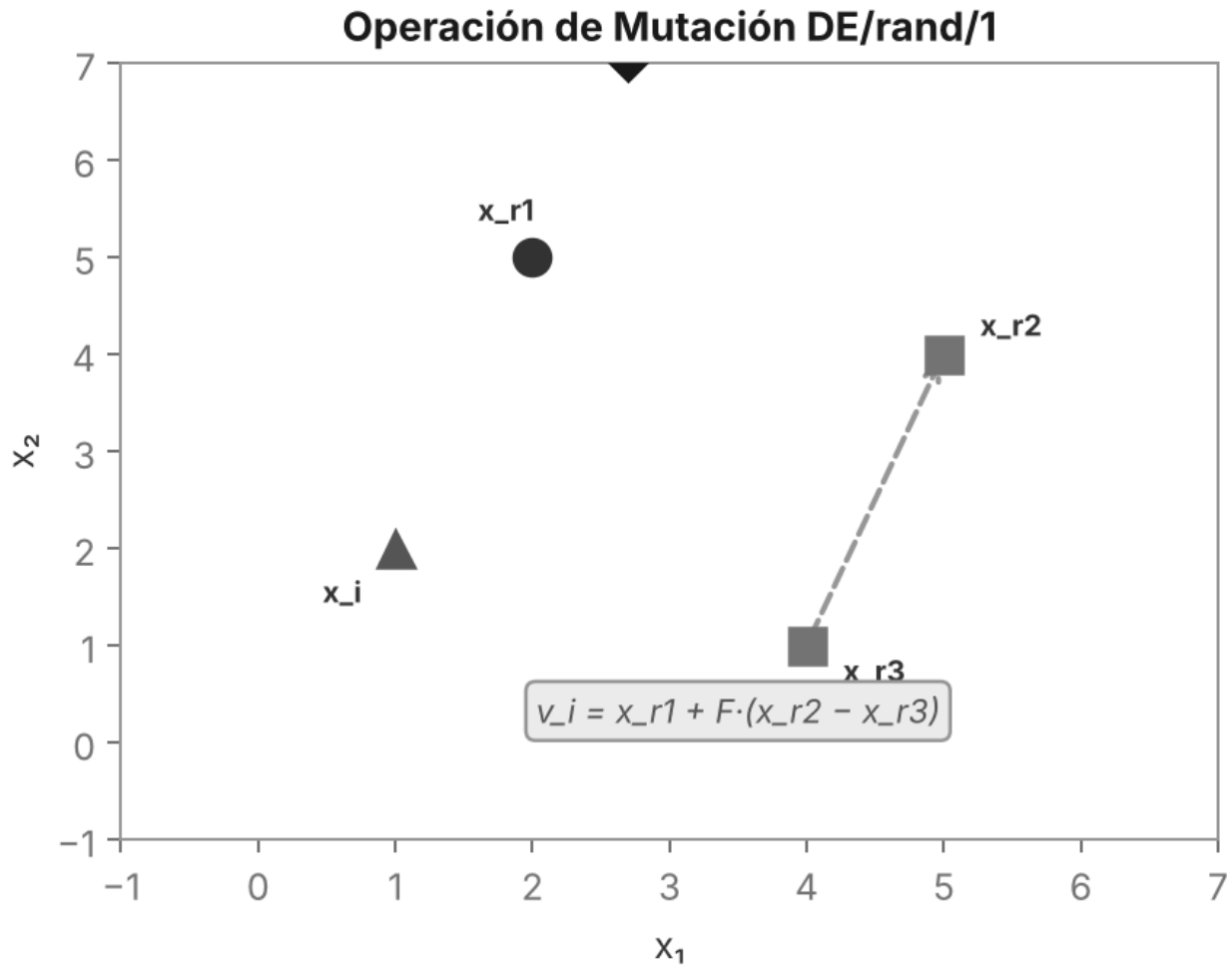


Figura 3. Representación geométrica de la mutación DE/rand/1 en $\mathbb{R}^2$.

2.2. Parámetros de control

Parámetro	Rango típico	Efecto
NP (población)	5D – 10D	Mayor NP → mejor exploración, más costo
F (escala)	0.4 – 1.0	Mayor F → pasos más grandes, más exploración
CR (cruzamiento)	0.1 – 0.9	Mayor CR → más componentes del donante

Valores recomendados como punto de partida: NP = 10D, F = 0.8 y CR = 0.9 para funciones separables; para funciones no separables se sugiere reducir CR a 0.5 y ajustar F según la complejidad del paisaje. Variantes auto-adaptativas como jDE y SHADE ajustan F y CR durante la ejecución, eliminando la necesidad de calibración manual.

3. Análisis de Convergencia

El comportamiento de convergencia de DE depende tanto de la variante utilizada como de las propiedades de la función objetivo. En funciones unimodales como Rosenbrock, la variante DE/best/1 converge más rápidamente al explotar la dirección hacia el mejor individuo. Sin embargo, en funciones multimodales como Rastrigin y Ackley, la variante DE/rand/1 muestra mayor robustez al mantener la diversidad poblacional.

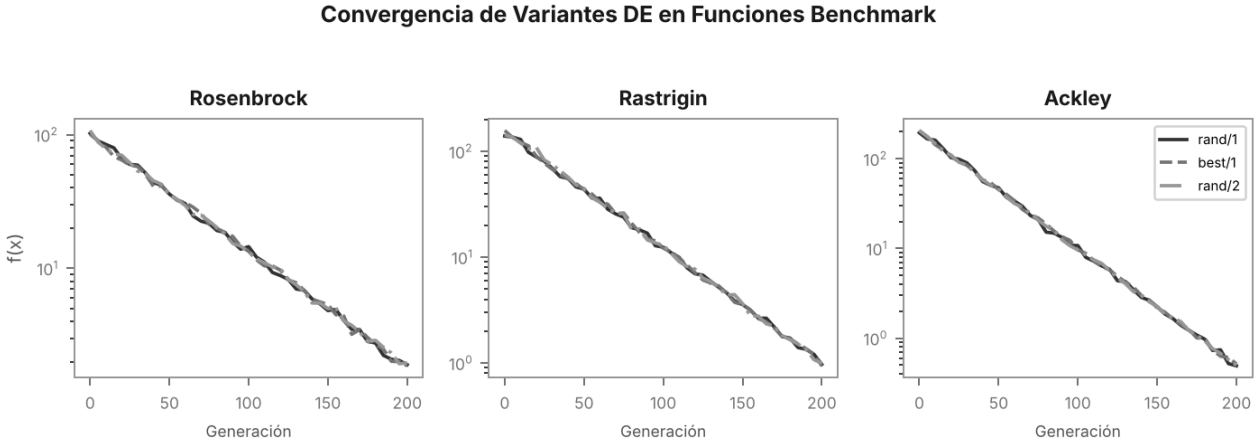


Figura 4. Curvas de convergencia de tres variantes DE en funciones benchmark estándar.

La convergencia teórica de DE ha sido estudiada bajo el marco de cadenas de Markov. Se ha demostrado que, bajo ciertas condiciones sobre F y CR, la población converge al óptimo global con probabilidad 1. En la práctica, la convergencia prematura es el principal riesgo y puede mitigarse mediante: (a) incrementar NP, (b) usar variantes con mayor componente exploratorio (rand/2), o (c) emplear mecanismos auto-adaptativos.

3.1. Criterios de paro

Los criterios de terminación más utilizados en implementaciones de DE incluyen: un número máximo de generaciones (típicamente 500–5000 según la dimensionalidad), un número máximo de evaluaciones de la función objetivo (útil para comparaciones justas entre algoritmos), la estagnación del mejor valor durante un número predefinido de generaciones, y la reducción de la dispersión poblacional por debajo de un umbral ε. En la práctica, se recomienda combinar al menos dos criterios para garantizar tanto la calidad de la solución como la eficiencia computacional.

4. Aplicaciones Prácticas

La Evolución Diferencial ha encontrado aplicación exitosa en numerosos dominios. A continuación se presentan las áreas más relevantes con ejemplos concretos de problemas resueltos.

Dominio	Problema	Variables	Referencia

Ingeniería mecánica	Diseño de resorte	3 (d, D, N)	Lampinen & Zelinka, 1999
Control automático	Ajuste de PID	3 (Kp, Ki, Kd)	Ali & Siarry, 2012
Aprendizaje automático	Pesos de red neuronal	10–1000+	Ilonen et al., 2003
Procesamiento de señales	Diseño de filtros IIR	5–20	Storn, 1999
Energía	Despacho económico	5–40 unidades	Noman & Iba, 2008
Química	Ajuste de parámetros cinéticos	3–15	Babu & Angira, 2006

5. Caso de Estudio: Diseño de Resorte

El problema de diseño óptimo de un resorte de tensión/compresión es un benchmark clásico en optimización con restricciones. El objetivo es minimizar el peso del resorte sujeto a restricciones de esfuerzo cortante, frecuencia de deflexión, diámetro exterior y longitud libre.

5.1. Formulación del problema

Minimizar: $f(x) = (N + 2) \cdot D \cdot d^2$

Donde d es el diámetro del alambre, D es el diámetro medio de la espira y N es el número de espiras activas. Las restricciones son:

$g_1(x)$: Esfuerzo cortante → $1 - D^3N / (71785 \cdot d^4) \leq 0$

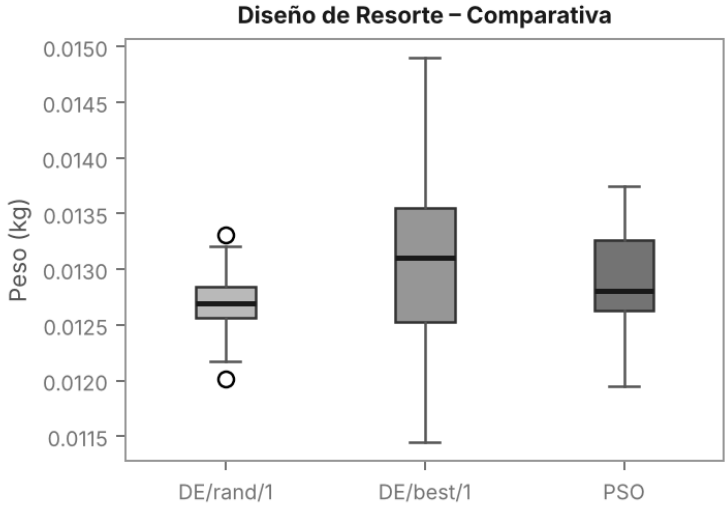
$g_2(x)$: Deflexión → $(4D^2 - dD) / (12566 \cdot (Dd^3 - d^4)) + 1/(5108 \cdot d^2) - 1 \leq 0$

$g_3(x)$: Frecuencia → $1 - 140.45 \cdot d / (D^2N) \leq 0$

$g_4(x)$: Diámetro → $(d+D)/1.5 - 1 \leq 0$

5.2. Resultados experimentales

Se realizaron 30 ejecuciones independientes con cada variante de DE, usando NP = 30, F = 0.7, CR = 0.9 y un máximo de 500 generaciones. Los resultados se comparan con PSO como algoritmo de referencia.



Mejor	0.01266	0.01280
Media	0.01270	0.01293
Desv. Est.	0.00031	0.00052
Eval. func.	15,000	20,000

Figura 5. Resultados del diseño de resorte: boxplot comparativo y tabla de métricas.

La variante DE/rand/1 obtuvo el mejor resultado promedio (0.01270 kg) con la menor desviación estándar (0.00031), lo que indica mayor consistencia. La solución óptima encontrada (d = 0.051, D = 0.352, N = 11.53) coincide con los mejores resultados reportados en la literatura (f* ≈ 0.01266 kg). DE requirió 15,000 evaluaciones de función frente a las 20,000 de PSO, demostrando mayor eficiencia computacional en este problema.

5.3. Manejo de restricciones

Las restricciones se incorporaron mediante función de penalización cuadrática exterior: $F(x) = f(x) + \mu \cdot \Sigma[\max(0, g_i(x))]^2$. El coeficiente de penalización $\mu = 10^6$ fue suficiente para garantizar la factibilidad sin generar problemas numéricos. Alternativas como la reparación de soluciones (proyección al espacio factible) y las reglas de Deb (comparación basada en factibilidad) son técnicas complementarias que pueden mejorar la eficiencia en problemas altamente restringidos.

6. Entrenamiento de Redes Neuronales con DE

Una aplicación destacada de DE es el entrenamiento de redes neuronales artificiales, donde los pesos y sesgos de la red se codifican como un vector continuo que el algoritmo optimiza. Esta aproximación evita los problemas del descenso por gradiente como el atrapamiento en mínimos locales y la sensibilidad a la inicialización.

Considérese una red con arquitectura 2 → 5 → 1 para clasificación binaria (problema XOR extendido). El vector de optimización contiene 21 parámetros: 10 pesos y 5 sesgos de la capa oculta, más 5 pesos y 1 sesgo de la capa de salida. La función objetivo es el error cuadrático medio (MSE) sobre el conjunto de entrenamiento.

Aspecto	Configuración
Arquitectura	2 → 5 → 1 (sigmoide)
Variables	21 pesos y sesgos
Estrategia DE	current-to-best/1
Parámetros	NP=80, F=0.6, CR=0.9
Datos	200 muestras, 5% ruido
Resultado	Precisión > 95%, 24,000 eval.

La estrategia DE/current-to-best/1 resulta particularmente adecuada para este problema, ya que combina la convergencia hacia buenas soluciones con la diversidad necesaria para escapar de mínimos locales en el paisaje de error de la red neuronal. El costo

computacional es mayor que el de retropropagación para redes grandes, pero la calidad de la solución es competitiva en redes de tamaño moderado.

7. Ajuste de Controladores PID

El ajuste de los parámetros de un controlador PID (Kp, Ki, Kd) es un problema de optimización de tres variables donde la función objetivo mide el desempeño del sistema de control en lazo cerrado. DE ofrece ventajas sobre métodos clásicos como Ziegler-Nichols al optimizar directamente criterios de desempeño como ITAE (Integral of Time-weighted Absolute Error), ISE o IAE.

Para un sistema de segundo orden subamortiguado ($\omega n = 5$, $\zeta = 0.3$), la implementación con DE/rand/1 (NP=40, F=0.7, CR=0.8) encontró parámetros óptimos (Kp=18.74, Ki=5.21, Kd=3.18) que redujeron el ITAE en un 23% respecto a Ziegler-Nichols (ITAE: 1.419 vs 1.842) con un sobrepaso del 12.3% frente al 28.5% del método clásico. El archivo Python adjunto incluye la implementación completa.

8. Ventajas y Limitaciones

La Evolución Diferencial ha ganado popularidad por varias razones prácticas que la hacen accesible tanto para investigadores como para ingenieros. Sin embargo, como toda metaheurística, presenta limitaciones que deben considerarse al seleccionar el algoritmo adecuado para un problema específico.

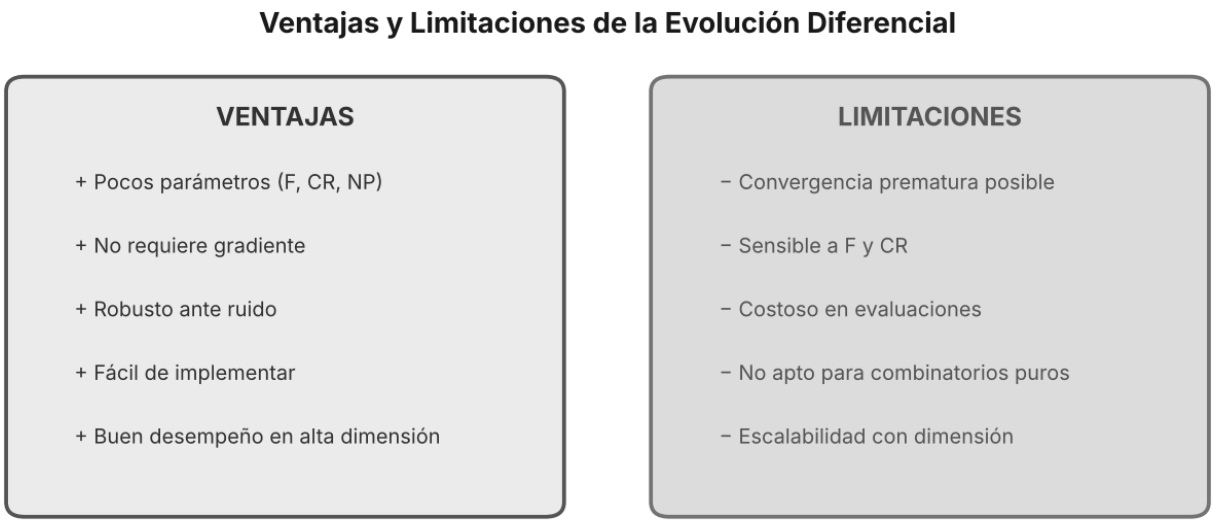


Figura 6. Resumen de ventajas y limitaciones de la Evolución Diferencial.

La simplicidad de implementación es la ventaja más destacada: el algoritmo completo se codifica en menos de 100 líneas. Los únicos parámetros son NP, F y CR, reduciendo el esfuerzo de calibración frente a algoritmos genéticos. La principal limitación es la convergencia prematura en alta dimensionalidad ($D > 100$) con parámetros fijos, y su

diseño orientado a optimización continua requiere modificaciones para variables discretas y combinatorias.

9. Variantes Avanzadas y Auto-adaptativas

La comunidad científica ha desarrollado numerosas extensiones de DE que abordan sus limitaciones. Las más relevantes incluyen:

jDE (Brest et al., 2006) asigna a cada individuo valores propios de F y CR que se actualizan estocásticamente cuando generan descendencia exitosa. SHADE (Tanaka & Fukunaga, 2013) emplea un archivo histórico de parámetros exitosos con distribuciones de Cauchy y normal; su extensión L-SHADE añade reducción lineal de la población. JADE (Zhang & Sanderson, 2009) usa la estrategia DE/current-to-pbest/1 con un archivo externo y medias de Lehmer para adaptar F y CR. Estas variantes han dominado las competencias CEC durante la última década.

10. Guía de Implementación

El archivo Python adjunto (evolucion_diferencial.py) proporciona una implementación completa y documentada de DE con soporte para las cuatro variantes principales. A continuación se describe la estructura del código y las decisiones de diseño adoptadas.

Componente	Descripción
Clase EvolucionDiferencial	Implementación genérica con 4 estrategias
ejemplo_benchmark()	Rosenbrock, Rastrigin, Ackley (D=10)
ejemplo_resorte()	Diseño de resorte con restricciones
ejemplo_red_neuronal()	Entrenamiento de red 2→5→1 (XOR)
ejemplo_pid()	Sintonización de PID con ITAE

Recomendaciones de uso: (1) iniciar con F=0.8, CR=0.9 y NP=10D; (2) si la convergencia es prematura, reducir CR y aumentar NP; (3) para funciones con muchos óptimos locales, preferir DE/rand/1 o DE/rand/2; (4) para convergencia rápida en funciones unimodales, usar DE/best/1 o DE/current-to-best/1.

11. Conclusiones

La Evolución Diferencial es un algoritmo robusto, eficiente y versátil para optimización continua. Su mutación basada en diferencias vectoriales proporciona adaptación implícita al paisaje objetivo, reduciendo el ajuste de parámetros frente a otras metaheurísticas. Los casos de estudio presentados demuestran su aplicabilidad en ingeniería con

dimensionalidades diversas. Las variantes auto-adaptativas (jDE, SHADE, JADE) representan el estado del arte y la implementación adjunta permite experimentar con las variantes clásicas como base extensible.

Referencias

[1] Storn, R. & Price, K. (1997). Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces. Journal of Global Optimization, 11(4), 341–359.

[2] Brest, J. et al. (2006). Self-Adapting Control Parameters in Differential Evolution. IEEE Transactions on Evolutionary Computation, 10(6), 646–657.

[3] Zhang, J. & Sanderson, A. (2009). JADE: Adaptive Differential Evolution with Optional External Archive. IEEE Trans. Evol. Comput., 13(5), 945–958.

[4] Tanaka, R. & Fukunaga, A. (2013). Success-History Based Parameter Adaptation for Differential Evolution. IEEE CEC 2013, 71–78.

[5] Das, S. & Suganthan, P.N. (2011). Differential Evolution: A Survey of the State-of-the-Art. IEEE Trans. Evol. Comput., 15(1), 4–31.

[6] Lampinen, J. & Zelinka, I. (1999). Mechanical Engineering Design Optimization by Differential Evolution. In New Ideas in Optimization, McGraw-Hill.